# V-OS

A graphical operating system for the Amstrad CPC 6128

Manual

Made from docs/manual/en/*.md and sdk/README.en.md by tools/mkdocs.py (make docs).

# Contents

V-OS is a graphical operating system for the Amstrad CPC 6128, written from scratch in Z80 assembly. It has:

- a menu bar, draggable windows, dialogs and buttons, used with the keyboard or an AMX mouse
- three views for files and programs (Desktop, Launcher, Commander) and the settings (Setup)
- six languages as plugins: English, Greek, French, Spanish, German and Italian
- programs that run side by side and resume where they left off, a desk accessory and programs in the background
- Studio, a Scratch-style block editor that runs on the CPC itself
- an API with 42 calls and an SDK for programs in assembly

This manual is also in Greek (../00-index.md), French (../fr/00-index.md), Spanish (../es/00-index.md), German (../de/00-index.md) and Italian (../it/00-index.md).


- 1. Getting started
- 2. The views
- 3. Programs
- 4. Languages
- 5. Studio: programs with blocks
- 6. Blocks on the PC: the web editor and `vbc.py`
- 7. The V-OS SDK: programs in assembly
- 8. Developing V-OS

# 1. Getting started

## What you need

- An Amstrad CPC 6128 or an emulator of it. V-OS is tested on Caprice32.
- The V-OS disc in drive A:, that is, the `build/vos.dsk` that `make` builds, and the programs' disc, `build/apps.dsk`, in B:.
- Optionally:
  - an AMX mouse
  - a second drive (B:)
  - a memory expansion (e.g. 512K), for more programs at once

## Starting up

From BASIC:

```
RUN"VOS
```

V-OS loads the kernel, switches off the ROMs and takes over the whole machine. It detects the memory expansion by itself.

- **First view:** the Desktop. You can change it in Setup, on the START VIEW line.
- **Language:** English. If there is a `SETUP.CFG` on the disc, it starts with the language and settings saved there.
- **Exit:** FILE > EXIT TO BASIC (CONTROL+Q in the views). It asks for confirmation and restarts the CPC.

## The screen

V-OS uses the whole screen (overscan): 384×272 pixels in mode 1 (4 colors) or 192×272 in mode 0 (16 colors).

- **Menu bar** (top): the menus of the view or program, usually FILE or the program's menu, VIEW and HELP. On the right is the keyboard language (EN, EΛ, ...).
- **Bottom line:** in each view, the keys that work there.

## Keys that work everywhere

| Key | What it does |
|---|---|
| F1 | Desktop |
| F2 | Launcher |
| F3 | Commander |
| F4 | Setup |
| F5 | opens and closes the desk accessory (ACCESSORY) |
| F6 | the running programs, to choose one |
| ESC | opens the first menu. In the menus, ←/→ change menu, ↑/↓ choose an item, RETURN runs it, ESC closes |
| CONTROL+letter | the menu shortcuts (e.g. ^Q, ^L) |

| | |
|---|---|
| CONTROL+L | switch language (VIEW > SWITCH LANGUAGE) |
| CONTROL+K | the dialog with the keys (HELP > KEYS) |
| CONTROL+SHIFT | switches the keyboard: English, the pack's, and, with KEYBOARDS: ALL SIX in Setup, the other languages' |
| TAB, SHIFT+TAB | move the focus between buttons. RETURN, ENTER or SPACE press the button that has the focus |

F1–F6 are the f1–f6 keys on the CPC's numeric keypad.

## The pointer and the mouse

- **With an AMX mouse:** the pointer moves with the mouse, and its button clicks. The mouse is switched on and set up in Setup (AMX MOUSE, AMX READING).
- **Without a mouse:** the pointer moves with the arrow keys, and COPY clicks. In views that use the arrow keys for choosing (Desktop, Launcher, Commander), the arrows choose files and programs instead of moving the pointer.

Most things work with both: one click on a file or program selects it, and a second click opens it.

## Windows and dialogs

- **Dragging:** every window can be dragged by its title bar: the dialogs, the Desktop's FILES window, the desk accessory, and the windows of block programs. While you hold the button a dashed outline is shown, and the window is redrawn where you let go.
- **Dialogs:** wait for an answer. TAB and ←/→ change the button, RETURN presses it, and ESC chooses the last one (usually NO or CANCEL).
- **Text input** (e.g. renaming): you type at the end, DEL deletes, RETURN accepts and ESC cancels. You can also type in the language of the pack.

## Discs and files

- **Format:** V-OS reads and writes AMSDOS discs (DATA or SYSTEM format) in A: and B:. BASIC can read its files too.
- **Names:** 8 characters plus 3 for the type (`NAME.TYP`).

Types you will come across:

| Type | What it is |
|---|---|
| `.BIN` | a V-OS program (or a binary file) |
| `.VBN` | a V-OS program made with blocks |
| `.VBS` | a Studio program (its source) |
| `.OVL`, `.MSG`, `.LNG` | parts of V-OS itself: views, texts, languages |
| `.SYS` | system files (e.g. `VBRUN.SYS`, the Studio player) |
| `SETUP.CFG` | the settings |

The settings, languages and views are read from A:, so the V-OS disc must stay there.

## Error messages

Disc errors are shown in a dialog: DRIVE NOT READY, WRITE PROTECTED, FILE NOT FOUND, DISC FULL, DIRECTORY FULL, NAME ALREADY USED, INVALID NAME and others.

Two more concern programs:

- **NOT A V-OS PROGRAM:** the file does not have the header of a program.
- **CLOSE ANOTHER PROGRAM FIRST:** there is no room. See chapter 3.

# 2. The views

V-OS has four "views": ways to see the files and programs, and the settings. You switch view with F1–F4 or from the VIEW menu. Each view remembers which file was selected, and the next one selects the same file.

## The common menus

The last two menus are the same in all views and in programs:

| VIEW | | HELP | |
|---|---|---|---|
| DESKTOP | F1 | KEYS | ^K |
| LAUNCHER | F2 | ABOUT V-OS | |
| COMMANDER | F3 | | |
| SETUP | F4 | | |
| PROGRAMS (running) | F6 | | |
| SWITCH LANGUAGE | ^L | | |

HELP > ABOUT V-OS shows the version (0.15 beta; each milestone adds 0.01), the date of the build, the maker (Revive8bit, 2026) and the architect (Vasper).

## Desktop, F1

Mode 1. A FILES window shows the files of a drive as icons, four in each row. The type is written inside the icon and the name below it, and a `.VBN` has a "play" mark next to it. On the right, two icons choose A: or B:. The title bar shows the disc format and the number of files, and the free space is shown at the bottom.

| Key | What it does |
|---|---|
| arrows | choose a file (↑/↓ one row) |
| TAB | the next drive: A:, B:, DISK (with an M4 board) |
| RETURN, ENTER | open: a program runs, a file of a known type opens in its program, others show their info |
| ^C | copy to the next drive (from DISK: to A:) |
| ^D | delete (asks first) |
| ^N | rename |
| ^R | read the discs again |
| mouse | click on a file: select, again: open. Click on a drive: opens it |

- **FILE menu:** OPEN, FILE INFO (^I), COPY TO NEXT DRIVE, DELETE, RENAME, READ DISCS AGAIN, EXIT TO BASIC.
- **The FILES window** can be dragged by its title and stays there, even when you switch view.
- **Opening by type** (also in the Commander): `.TXT` files open in NOTEPAD, `.MD` in MARKDOWN, `.PIC` and `.LYR` in PAINT, `.VDB` in DATABASE, `.VSH` in SHEET, `.MUS` in MUSIC, which starts playing them. The program is looked for on the file's drive, then on A:, then on B:. If it is not found, "NO PROGRAM FOR THIS FILE" is shown. If the program has unsaved changes, it asks first.

## Launcher, F2

Mode 0, 16 colors. Shows only the programs of one drive (`.BIN`, `.VBN`, `.BAS`), A: at first, as tiles in a

3×3 grid, with the next pages below. The page line starts with the drive (`A:`, `B:`). Every program that has an icon shows it on its tile. The selected tile is white.

| Key | What it does |
|---|---|
| arrows | choose (↑/↓ one row. Past the edge, the next or previous page) |
| RETURN, ENTER | runs the program. If it is already running, it resumes where it was |
| ^A | loads it as the desk accessory (AS ACCESSORY). F5 then opens it |
| ^R | read the disc again |
| TAB | the next drive (NEXT DRIVE): A:, B:, DISK. A program started from B: comes back to B: |
| mouse | click: select, again: run |

**FILE menu:** RUN, AS ACCESSORY, NEXT DRIVE, READ DISCS AGAIN, EXIT TO BASIC.

## Commander, F3

Mode 1, two panels with file lists: A: on the left, B: on the right. Each panel shows the format and the number of files in its title, and the free space at the bottom. The active panel has a dark title and a selection.

| Key | What it does |
|---|---|
| ↑/↓ | choose (with SHIFT: one page) |
| TAB, ←/→ | the other panel |
| RETURN, ENTER | open, as in the Desktop. The info (size, type, load address, the first bytes) with ^I |
| ^C | copy to the other panel, of any size. If there is a file with the same name, it asks first |
| ^D | delete |
| ^N | rename |
| ^R | read both discs again |
| ^G | the next drive for the active panel: A:, B:, DISK (also a click on its title) |
| mouse | click: select, again: open |

**FILE menu:** OPEN, FILE INFO (^I), COPY TO OTHER PANEL, DELETE, RENAME, READ DISCS AGAIN, NEXT DRIVE (^G), FORMAT DISC (DATA), EXIT TO BASIC.

> FORMAT erases all the files on the disc of the active panel. It asks first.

## DISK: the M4 board's SD card

With an M4 board, the Desktop has a third drive icon, DISK, under A: and B:, and TAB goes A: → B: → DISK. The Launcher and the Commander's panels (^G) reach it too. The title shows the folder you are in, e.g. `DISK:/GAMES/`.

- **Folders** show as folders (an icon with a tab, or DIR in the Commander). RETURN on one goes into it, and `..`, the first entry, goes back up. In the Launcher, folders are there too, so that you can reach the programs inside them.
- **Everything as on a disc:** programs run from it, files open in their programs, and ^C, ^D and ^N copy, delete and rename. Copies between DISK and a disc go up to 16K. Folders and files are named as on the CPC (8.3: `NAME.EXT`); names that are longer do not show.
- **M4.SYS** (on `vos.dsk`) is loaded from A: the first time DISK is used.
- **Not yet:** the programs' OPEN and SAVE (NOTEPAD, PAINT...) still see only A: and B:. DATABASE works only on discs, and the SD card is not formatted from V-OS. V-OS itself still starts from a disc in A: (type `|DISC` before `RUN"VOS"`: with an M4 board, the CPC starts on the

SD card).

## Setup, F4

Mode 1. One line for each setting. Changes take effect at once. When saved (RETURN or ^S) they are written to `A:SETUP.CFG` and also apply at the next start-up.

| Setting | Values |
|---|---|
| START VIEW | DESKTOP, LAUNCHER, COMMANDER |
| LANGUAGE | ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO |
| REPEAT DELAY | how long you hold a key before it starts repeating: SHORT, MEDIUM, LONG |
| REPEAT SPEED | SLOW, MEDIUM, FAST |
| AMX MOUSE | ON, OFF |
| AMX READING | LEVELS (at first) or PULSES: how the mouse is read. LEVELS counts every read with movement, PULSES only the starts of the pulses. If the pointer moves too fast or too slowly, try the other one |
| DESK COLOUR | BLUE, BLACK, GREEN, RED |
| KEYBOARDS | EN + PACK, ALL SIX (the ones CONTROL+SHIFT goes through) |

| Key | What it does |
|---|---|
| ↑/↓ | choose a setting |
| ←/→ | change the value (also a click on the line) |
| RETURN, ^S | save |

Changing the LANGUAGE loads the language pack from A: at once. See chapter 4.

# 3. Programs

## Starting and ending

A program starts in one of these ways:

- from the Launcher: select it and press RETURN, or click twice
- from the Desktop or the Commander: FILE > RUN
- from another program, such as Studio, which runs whatever you make

Each program has its own menu next to the common VIEW and HELP. It ends from its menu (usually BACK or STOP, CONTROL+Q) and returns to the view it was started from.

## Several programs at once

V-OS programs are of two kinds:

- **Small, with a relocation table:** DEMO, MIXED and CLOCK, the web editor examples (PONG, CATCH, STARCAT, COUNTER), and whatever the SDK makes. They share 12K of memory, and as many as fit run together.
- **Programs that need all the memory:** STUDIO and whatever Studio makes (CALC, MINES, FLAP, TICKER and your own). So are MARKDOWN and READER, which are over 12K. Without a memory expansion they run on their own. If one is already running, you get "CLOSE ANOTHER PROGRAM FIRST".

When you go to a view (F1–F4) while a program is running:

- A small program stays in memory. You pick it up again from the Launcher (RETURN on it) or with F6, and it resumes exactly where it was.
- A program that has all the memory ends, unless there is a memory expansion.

**F6** (VIEW > PROGRAMS) shows the list of running programs. You choose one with the arrows and RETURN. If none is running, you get "NO PROGRAM IS RUNNING".

**With a memory expansion** (e.g. 512K, which V-OS finds by itself at start-up), every program that needs all the memory gets its own bank. So STUDIO, CALC, DEMO and PONG can all run together. Since M43 every program also gets a data bank of its own, 16K, when it needs one: what it keeps there is not lost while other programs or views run.

## In the background

A program behind another one, or behind a view, normally **freezes** and resumes when it comes back to the front.

Some programs ask to **keep running** in the background too, e.g. TICKER, which counts time. While there they do not draw or play sound. When they come back to the front, they have also counted the time they were away.

In Studio you choose this for your program with STUDIO > IN BACKGROUND.

MUSIC is the exception: its song plays from the interrupt, 50 times a second, so it goes on evenly behind everything. It stops only for a moment while the disc is read.

## The desk accessory (ACCESSORY)

A small program, such as CLOCK, can open in a window **on top of anything**: a view or a program.

1. In the Launcher, select CLOCK and choose FILE > AS ACCESSORY (^A).
2. From then on, **F5** opens and closes it, wherever you are. ESC also closes it.

While it is open, it gets the keys and clicks. Its window can be dragged by its title. When it closes, whatever was underneath is redrawn.

## The page (mode 2)

NOTEPAD, MARKDOWN, READER and SHEET have PAGE (MODE 2) in their menu, ^M. The screen goes to mode 2: black text on white paper, 768 pixels across, with the menus and dialogs black on white too. The editors (NOTEPAD and MARKDOWN) show 80 columns between the page's edges, a line where each page of 60 lines ends, and PAGE n in the last line. MARKDOWN's preview and READER show the page in 80 columns with its styles. SHEET shows 10 columns (A–J) with lines between them. ^M again goes back to mode 1.

## The programs on the disc

A CPC disc holds 180K, so V-OS comes on two. `vos.dsk` (A:) has the system, DEMO, MIXED, STUDIO and CLOCK, and the `.VBS` sources of the VB programs, which Studio opens from A:. `apps.dsk` (B:) has the others, from PONG to WEBGET: in the Launcher, TAB shows B:. Some programs have a disc of their own, as the table says.

| Program | What it is |
|---|---|
| **STUDIO** | the block editor: you make programs on the CPC (chapter 5) |
| **DEMO** | the first V-OS program: a square that moves by itself. The arrows steer it and SPACE changes its color |
| **MIXED** | the "mixed screen": a band with 16 colors (mode 0) between mode 1 bands. A click shows the coordinates |
| **CLOCK** | the time since V-OS started. It also runs as a desk accessory |
| **CALC** | a calculator in a window with buttons, made in Studio (there is also `CALC.VBS`) |
| **MINES** | an 8×6 minesweeper with 8 mines, made in Studio. A click opens a cell, and NEW starts a new game |
| **BRICKS** | brick breaking: a paddle with ←/→, 32 bricks, 3 lives, levels with an ever faster ball. At GAME OVER, SPACE for a new game |
| **VROUVES** | tower defence, from the CPC game LET'S GO FOR VROUVES: 7 stages, from the Kafenio to the cemetery, with 10 waves each. It is on its own disc, `vrouves.dsk`: put it in B: and press TAB in the Launcher. Arrows move the cursor, 1–6 picks a defender, SPACE builds, S sells, N calls the next wave, P pauses. Points for every invader and for the villagers left at the end of a stage. The disc keeps `VROUVES.SAV`: CONTINUE starts from the stage you reached, HIGH SCORES shows the best five with their names. Written with the SDK (`myapps/vrouves`) |
| **NOTEPAD** | a text editor in mode 1, 48×22, for files up to 4K, or up to 16K with a memory expansion: then it keeps about 2.5K around the cursor in its memory and the rest in its own data bank, and SELECT ALL takes that part. NEW, OPEN (only `.TXT` files), SAVE, SAVE AS, OTHER DRIVE. Arrows, DEL, CLR and RETURN; a click places the cursor. The bottom line shows the line and the column. EDIT menu: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT and the arrows select. The clipboard (up to 1K) is shared by all programs (`myapps/notepad`) |

| | |
|---|---|
| **PAINT** | drawing in mode 0 with 16 colors: a picture of the whole CPC screen, 160×200 without the overscan (`.SCR`, 16K, which BASIC and other CPC programs load too; without a memory expansion 128×128, `.PIC`), or 64×64 at 2× zoom in 3 layers (`.LYR`, NEW LAYERS). Tools: P pencil, B brush, L line, X box, F fill; TAB changes the color. Arrows and COPY draw without a mouse. The window shows 128×128 of the picture: the bars on its right and under it show which part, and a click or a drag on them moves it. Under the colors: where the pointer is in the picture, and VIEW, the window's top left. Layers: 1–3 choose one, H hides or shows it, U and D move it up and down, E is the eraser. EXPORT .SCR (for the 128×128 picture and the layers; the whole screen is a `.SCR` already): the picture (the layers as they show) as a CPC screen, a 16K `NAME.SCR` (mode 0, in the middle, with the palette), for BASIC or other programs. In BASIC: `MODE 0:LOAD"NAME.SCR",&C000:FOR I=0 TO 15:INK I,PEEK(&C7D0+I):NEXT:BORDER PEEK(&C7E0)` (`myapps/paint`) |
| **MUSIC** | plays Arkos Tracker 3 songs (`.MUS`, made by `make songs`), with three volume bars. It is on its own disc, `music.dsk`, in B:. OPEN, SPACE plays and stops. The music goes on behind other views and programs, evenly, even while another program loads or draws: it plays from the interrupt. Studio's MUSIC block starts it behind a program with a song (`myapps/music`) |
| **MARKDOWN** | an editor for Markdown files (`.MD`, up to 6K), like NOTEPAD. ^P shows the page: `#` headings, **bold**, *italic*, `code`, lists, quotes, lines, code blocks and underlined links, without the marks. Lines that follow each other make one paragraph (two spaces at the end break the line). The arrows scroll the page, RETURN or ^P go back to the text. It is on its own disc, `markdown.dsk`, with a `README.MD`. EDIT menu: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT and the arrows select. The clipboard (up to 1K) is shared by all programs (`myapps/markdown`) |
| **SYSINFO** | shows the V-OS and API versions, the memory (all of it and what is free for programs) and the free space on the discs in A: and B: (^R again). An example of `VOS_EXT` (`myapps/sysinfo`) |
| **WEBGET** | downloads files from the web with the M4 board: GET A FILE (^G) asks for an address (HTTP, e.g. `example.com/index.html`) and a name, and the file (up to 16K) is saved on the drive it shows (OTHER DRIVE). It also shows the network status. Without an M4 board: "NO M4 BOARD" (`myapps/webget`) |
| **READER** | reads Markdown pages (`.MD`, up to 6K) from the disc or from the web with the M4 board, shown as in MARKDOWN's preview. TAB selects the next link (inverted), RETURN opens it, DEL goes back to the previous page (it keeps 4). A link without `http://` opens next to the page, on the disc or on the same site. OPEN A FILE and OPEN AN ADDRESS (^U, HTTP, e.g. `example.com/README.md`). `https://` addresses cannot be opened. It is on its own disc, `reader.dsk`, with `INDEX.MD` and `ABOUT.MD` (`myapps/reader`) |
| **STREETS** | a street index in a V-OS database: type the first letters of a street and see the streets that start so, with their area and post code; DEL takes a letter back. ^A adds a street, ^D deletes the first one in the list. The streets are in `STREETS.VDB` on its disc, `streets.dsk`, in B:; the first time it makes it, with streets of Athens. An example of the database API (`myapps/streets`) Finding ignores capitals, small letters and accents. |
| **DATABASE** | databases, like a small Access: ^N a new database (`NAME.VDB`), ^O open one, ^T a new table (its name, and fields: a name, TEXT up to 40 characters or NUMBER 0–65535, and whether it is sorted by the first one). The list shows a record a row and the fields in columns; the arrows choose and scroll, TAB goes to the next table, and if the table has a key, what you type goes to the records that start so (without case or accents). RETURN opens the form: a field a row, RETURN changes it, ^S saves, ^B back to the list. ^A a new record, ^D delete, ^W the other drive. It opens STREETS's `.VDB` too (`myapps/database`) |

| | |
|---|---|
| **SHEET** | a spreadsheet: 16 columns (A–P) and 64 rows, 5 columns and 19 rows on the screen. A cell holds, up to 40 characters, a number (`12`, `-3.5`; the decimal point is a dot), text, or a formula after `=`: `+ - * /`, brackets, cells (`B3`), SUM, AVG, MIN, MAX and COUNT of cells and ranges (`=SUM(B2:B9)`), ABS, INT, ROUND(x, 0–2). Numbers have two decimals, up to ±21474836.46. The arrows move the cursor, RETURN changes the cell, typing starts a new text, DEL (or CLR) empties it; after each change all the formulas are worked out again. The first line shows the cell's text. A cell with an error shows `#ERR`, a number that does not fit `########`. ^N new, ^O open, ^S save, ^A save as (`NAME.VSH`, a text file, a line a cell: `B3 =B1*2`), ^W the other drive. The formulas are worked out by `CALC.SYS` on A:, which any program can use (`VX_CALC`). It is on its own disc, `sheet.dsk`, with the example `BUDGET.VSH` (`myapps/sheet`) |
| **PONG** | a ball and a paddle with the arrows, score and lives |
| **CATCH** | catch the falling stars |
| **STARCAT** | a cat with costumes that walks, a star, a melody from a list |
| **COUNTER** | a counter with +1, −1 and 0 buttons, in mode 1 |
| **FLAP** | a bird that flaps its wings (costumes), and a star that changes costume every second |
| **TICKER** | counts seconds and keeps going in the background |

- Block programs are `.VBN` files. They have a PROGRAM > STOP menu (CONTROL+Q).
- Those that also have a `.VBS` (CALC, MINES, FLAP, TICKER) open in Studio so you can look at them or change them.
- `VBRUN.SYS` is not a program: it is the "player" that Studio needs.

# 4. Languages

V-OS speaks six languages. English is built in. The other five are **language packs**, files on the disk that bring three things: all the system texts, the keyboard, and the letters (font) of the language.

| Language | File | Keyboard |
|---|---|---|
| English | built in (`ENGLISH.MSG`) | the CPC's English one |
| Ελληνικά | `GREEK.LNG` | Greek, with accents |
| Français | `FRENCH.LNG` | AZERTY, digits with SHIFT, dead keys ^ and ¨ |
| Español | `SPANISH.LNG` | Ñ on the `:` key, dead keys ´ ¨ ` ^, and ¡ ¿ |
| Deutsch | `GERMAN.LNG` | QWERTZ, Ü Ö Ä ß, dead keys ´ and ` |
| Italiano | `ITALIAN.LNG` | è é ò à ù ì on their own keys |

## Switching language

- **Permanently:** in Setup (F4), the LANGUAGE line cycles through all the languages. Each one loads from A: at once, and the whole screen is redrawn in it. Saving (^S) keeps it for the next startup too.
- **Quickly:** CONTROL+L (VIEW > SWITCH LANGUAGE) switches between English and the language of the pack that is loaded.
- **Keyboard only:** CONTROL+SHIFT switches the keyboard. The menu bar shows on the right which one is active (EN, EΛ, FR, ...). With Setup's KEYBOARDS at EN + PACK it goes between the English one and the pack's; at ALL SIX it goes through all of them: EN, the pack's, the other four, and EN again. So you can type French with the French keyboard while the system is in Greek. Every pack holds the keyboards of all of them.

The symbols `@ # [ ] { } \ |` are not on the packs' keyboards, because the CPC has no AltGr. Type them with the English keyboard.

## Program texts

Programs have their own texts in **two languages**: English and, optionally, one more (today, Greek). A program appears in its second language only when the pack is in that language. Otherwise it appears in English. So, with French, Studio appears in English. Studio's programs, though, can have their texts in all six languages (chapter 5): CALC is called "CALCULATRICE" in French.

## One alphabet for all

All languages write with the same character set, the **V-OS set**, and every pack has all its letters. So a text or a database can mix Greek, French, German, Spanish and Italian, and they show right whatever language is active: in Setup, LANGUAGE shows ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL with their own letters.

Codes 128 to 255 are the Greek ones of ISO 8859-7, as they were, and the codes it leaves free hold À Á Â Ä Ç È É Ê Ë Ì Í Î Ï Ñ Ò Ó Ô Ö Ù Ú Û Ü, their small letters, ß and ¡ ¿. The ´ is written as the Greek tonos ( ΄ ) and µ as µ. There is no ÿ, Œ or œ. The table is in `tools/voscs.py`. A Greek text file from a PC (ISO 8859-7) reads as it is; a French or German one in Latin-1 needs converting.

With the English keyboard, SHIFT and `^` gives £.

Files in **UTF-8**, from a PC or the web, are opened by NOTEPAD and READER, which turn them into

the V-OS set; what it has not (Chinese, say) becomes `?`. On the PC, `tools/voscv.py` converts files to and from V-OS (UTF-8, Latin-1 and others) and says what a file is (`check`).

## Making a new language

The language sources are in `lang/<code>/`:

- `messages.txt`: the system texts. The base list is defined by `lang/en/messages.txt`, and anything missing from a pack appears in English.
- `keymap.txt`: the keys that differ from the English keyboard.
- `pack.txt`: the code, the name, the character set and the font.

`tools/mklng.py` turns them into a `.LNG`, and the Makefile puts it on the disk. All packs have the same font, `lang/vos/font.txt`, and `charset vos`.

# 5. Studio: programs with blocks

STUDIO is a Scratch-style block editor that runs on the CPC itself.

- You make sprites and scripts out of blocks.
- The program runs at once with CONTROL+R.
- It becomes a normal V-OS program (`.VBN`) that also starts from the Launcher.

## The screen

Studio uses a "split" screen, with two zones:

- **Top, in mode 0 (16 colors):**
  - **left, the stage** at half size
  - **right, the SPRITES panel:** the sprites with their color and position, the size of the selected one (W, H), the background and the program's mode (e.g. `STAGE 2 M1`), and the file name
- **Bottom, in mode 1:**
  - **left, the palette** with the blocks of one category (`< EVENTS >`)
  - **right, the script**, one line per block
  - **at the end, the status line**, with the keys that work where you are

**TAB** goes from the script to the palette, then to the stage, and back to the script.

## Menus

| STUDIO | | EDIT |
|---|---|---|
| NEW | ^N | DELETE LINE |
| OPEN (`.VBS`) | ^O | + SPRITE (up to 6) |
| SAVE | ^S | − SPRITE |
| SAVE AS | | BACKGROUND (the stage's color) |
| RUN | ^R | SCREEN MODE (0 or 1) |
| IN BACKGROUND | | TEXT LANGUAGE (^T) |
| BACK | ^Q | |

- **RUN (^R):** saves `NAME.VBS`, writes `NAME.VBN` and runs it. The program stops from its own menu (PROGRAM > STOP, ^Q), and then Studio starts again with the same program open.
- **IN BACKGROUND:** asks whether the program keeps running when it is behind another one or behind a view, or pauses, which is the default.
- **SCREEN MODE:**
  - **Mode 0:** 16 colors, 192 pixels wide.
  - **Mode 1:** 4 colors (pens 0–3), 384 pixels wide, and more text.

## The palette

←/→ change the category, ↑/↓ choose a block, and RETURN adds it below the script line. A click on a

palette block does the same.

A block that opens (FOREVER, REPEAT, IF) goes in together with its END, and is deleted together with it. A block with text asks for the text at once.

## The script

| Key | What it does |
|---|---|
| ↑/↓ | line |
| ←/→ | slot of the line (the slots are the values inside the block) |
| RETURN | with no slot selected: adds the palette block. On a text slot: change the text. On a key slot: waits for you to press the key |
| CLR | deletes the line |
| DEL | deletes the last digit of a number |
| mouse | click on a slot: selects it. Second click: changes its kind |

The program is a list of lines:

- Each `WHEN …` starts a script, for a sprite or for the stage. The script runs up to the next `WHEN`.
- The indentation comes by itself from the blocks that open and close.
- Lines before the first `WHEN` are ignored.
- Anything left open is closed at the end of the script.

### What goes in a slot

The status line always says what the slot accepts.

**Value slot:**

| Key | Value |
|---|---|
| 0–9, − | number, an integer from −32768 to 32767 |
| A–H | variable. The same letter again: A1, A2 … A15, and back to A |
| X, Y | the position of the script's sprite |
| R and digits | RANDOM: a number from 1 to n |
| K and a key | KEY: 1 while it is held down, otherwise 0 |
| T and a digit | TOUCHING: 1 when the sprite touches sprite n |
| Z | AT EDGE: 1 when the sprite is at the edge of the stage |
| P | BUTTON: the number of the last button pressed |
| U | COSTUME: the number of the sprite's costume (0: no costume) |

**Other kinds of slot:**

- **Variable:** A–H, and the same letter again for A1…A15.
- **Comparison:** `<`, `=`, `>`. SPACE gives "none", and then IF only checks that the first value is not 0.
- **Sprite** (in the `WHEN` blocks): 0 the stage, 1–6 a sprite.
- **Key** (in `WHEN KEY`): RETURN and then the key.
- **Text:** RETURN, and you type up to 14 characters.
- **Variable group** (SAVE/LOAD VARS): A–H moves to the next group of eight (A-H, A1-H1 … A15-H15), and SPACE gives ALL.

The letters are read as keys, so they also work with the Greek keyboard.

## The stage and the sprites

| Key | What it does |
|---|---|
| arrows | move the selected sprite |
| 1–6 | select a sprite |
| C | next color |
| W, H | width, height |
| K | which costume it wears (none, 1, 2, ...) |
| E | draw its costume (a new one, if it has none) |
| click on the stage | puts the sprite there |

A sprite without a costume is a colored rectangle.

### Costumes

**E** opens the costume editor in place of the stage. It has a 16×16 grid with the 16 pens below it.

| Key | What it does |
|---|---|
| arrows | cell |
| SPACE, RETURN | paints the cell |
| 0–9, A–F | pen |
| . | transparent |
| E, TAB | done |
| click | on a cell: paints. On the row of pens: selects a pen |

A program has up to 8 costumes, shared by all sprites. The size of a sprite with a costume is 16×16. On the stage, x is rounded to a whole byte (2 pixels in mode 0, 4 in mode 1).

## All the blocks

In the "Block" column, the slots appear as in the script.

### EVENTS

| Block | What it does |
|---|---|
| WHEN START *sprite* | the script starts together with the program |
| WHEN KEY *key sprite* | starts when the key is pressed |
| WHEN CLICKED *sprite* | starts with a click on the sprite (or on the stage) |
| WHEN I RECEIVE *n sprite* | starts when message n is sent |
| SEND *n* | sends message n (n can also be a variable) |

> A message starts **from the beginning** every script that waits for it, even if it was already running. That is why a script should not send a message to itself.

### CONTROL

| Block | What it does |
|---|---|
| FOREVER ... END | repeats forever |
| REPEAT *n* ... END | n times |
| IF *a comparison b* ... [ELSE ...] END | runs the lines only if the comparison holds |
| WAIT *n* | waits n frames (50 = one second) |

| | |
|---|---|
| WAIT UNTIL *a comparison b* | waits until the comparison holds |
| STOP | ends this script |

## MOTION

| Block | What it does |
|---|---|
| GO TO *x y* | the sprite goes to x, y |
| CHANGE X *n*, CHANGE Y *n* | moves by n |
| SPEED *vx vy* | sets its speed |
| MOVE | one step at its speed |
| BOUNCE | at the edge of the stage, reverses the speed |

## LOOKS/SOUND

| Block | What it does |
|---|---|
| COLOUR *n* | the rectangle's pen |
| COSTUME *n* | wears costume n (1–8. 0 changes nothing) |
| NEXT COSTUME | the next one, and after the last one the first again |
| SHOW, HIDE | |
| PRINT *value* AT *x y* | prints a number |
| STYLE *n* | the style of the PRINT and TEXT that follow: 1 bold, 2 italic, 4 underline, added together (7: all), 0 plain |
| NOTE *n* FOR *frames* | plays a note (MIDI number: 60 = C, 69 = A 440 Hz) and waits for it to finish |
| MUSIC *"name"* | plays the song *name*`.MUS` from the program's disc, behind it, with the MUSIC program (on A: or B:). It goes on after the program ends, until MUSIC's STOP (F6). It needs a memory expansion; without one: "CLOSE ANOTHER PROGRAM FIRST". A name that is not there (or empty) stops the music |

## VARIABLES

| Block | What it does |
|---|---|
| SET *variable* TO *value* | |
| CHANGE *variable* BY *value* | addition |
| DECREASE *variable* BY *value* | subtraction |
| MULTIPLY *variable* BY *value* | |
| DIVIDE *variable* BY *value* | integer division |
| REMAINDER *variable* BY *value* | the remainder of the division |
| SAVE VARS *group* TO *"name"* | writes the variables to a text file (see below) |
| LOAD VARS *group* FROM *"name"* | reads them back |

## WINDOWS

| Block | What it does |
|---|---|
| WINDOW *"title" width height* | a window in the middle of the stage, like the V-OS dialogs. A new window takes the place of the previous one and its buttons |
| BUTTON *"text"* AT *x y* | a system button. It is pressed with a click or with TAB and RETURN. Buttons are numbered from 0, in the order they appear |

| | |
|---|---|
| WHEN BUTTON *sprite* | starts when any button is pressed. The BUTTON value (P) says which |
| TEXT *"text"* AT *x y* | writes text |
| LABEL *button* TO *"text"* | changes the text of a button |

With a window open, TEXT, BUTTON and PRINT count from the top-left corner of its inside. The window is dragged by its title, together with its buttons.

## Variables

There are 128 variables, shared by all scripts, named A–H, A1–H1, ..., A15–H15. At the start they are all 0, and they hold integers from −32768 to 32767.

### Saving and restoring (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` writes all 128 to the file `GAME.TXT`. The `.TXT` is added when the name has no type.
- With a group (`A-H`, `A1-H1`, ...) it writes only those 8.
- The file is text, one line per variable:

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` changes the variables the file mentions. With a group, it changes only those that are also in the group. Lines it does not understand are ignored, and if the file is missing nothing changes.
- So a file can restore all the variables or just one group of eight, e.g. a game's saved positions.

## Texts in other languages

A program's texts (windows, buttons, TEXT, LABEL) can have a translation in each language of V-OS. The program shows them in the language V-OS is in when it starts. So CALC is called "ΑΡΙΘΜΟΜΗΧΑΝΗ" in Greek, "CALCULATOR" in English, "CALCULATRICE" in French.

- **EDIT > TEXT LANGUAGE (^T):** goes through the languages EN, EL, FR, ES, DE, IT and back to the program's own texts. Under the sprites, next to the name, you see the language's code (e.g. `CALC FR`).
- In a language, the text slots show and change the texts in it. A text with no translation shows its own, and when you change it, that becomes its translation. A new text that you write in a language also becomes its own.
- The translations are saved inside the `.VBS` (and the `.VBN`), all six together, about 3K.
- In a `.vbt` they are written with `LANG FR` followed by lines `"text" "translation"`.

## Files

- **`NAME.VBS`:** the program as you edit it. OPEN opens it, and up to 9K is read.
- **`NAME.VBN`:** the program ready to run. RUN writes it, and it also starts from the Launcher. It is the player (`VBRUN.SYS`) together with the program. The player compiles the blocks to Z80 code when it starts, so programs run fast.

A `.VBS` can also be made from text on the PC, with `tools/mkvbs.py`. The syntax is described at the

top of the file, and examples are `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS). CALC and MINES also have translations (`LANG`).

## Limits

| | |
|---|---|
| Lines (blocks) | 240 |
| Sprites | 6 |
| Scripts (`WHEN …`) | 16 |
| Variables | 128 |
| REPEAT in one program | 64 (beyond that they share a counter) |
| Open blocks nested inside each other | 16 |
| Texts | 32, up to 14 characters each, and in 6 languages |
| Costumes | 8, 16×16 |
| Buttons | 64 |
| Code | at least 4,000 bytes, even for the largest program. Whatever does not fit is ignored |

## Tips

- **Stage scripts:** the motion and looks blocks do nothing, and X, Y, TOUCHING and AT EDGE give 0.
- **TOUCHING:** holds only when both sprites are visible.
- **Timing:** scripts run "together". Each script gives up its turn at the end of every loop and at every WAIT. To make one script wait for another, use a variable and `WAIT UNTIL` (see MINES).
- **Ready-made examples:** CALC, MINES, FLAP and TICKER open in Studio (OPEN).

# 6. Blocks on the PC: the web editor and `vbc.py`

Besides Studio, which runs on the CPC, there is also a block editor for the PC. The language is the same, with a few extra blocks: lists, noise, sprite size, colored text.

## The editor

Open `blocks/editor.html` in a browser. It is based on Blockly and comes in Greek or English.

- **Categories:** events, control, motion, looks, text, operators, sensing, variables, lists and windows.
- **Sprites:** rectangles or costumes. They are defined in a table, and the stage shows them in the CPC's colors. They can be dragged with the mouse.
- **Files:** it opens and saves `.vbp` files. They are JSON with the whole program, so they can also be written by hand.

## Compiling

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` writes `build/NAME.ASM`, with a comment for each block. It assembles it with RASM into `NAME.VBN`, with a relocation table so that it runs alongside other programs, and with `--dsk` it puts it on the disk.

| Option | What it does |
|---|---|
| `-o build` | output folder |
| `--asm-only` | only the `.ASM` |
| `--dsk build/vos.dsk` | also onto the disk |

## The `.vbp` format

The full description is at the top of `blocks/vbc.py`: events, commands, values, costumes, icon. In short:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variables and lists:** they have names, not letters. Lists have a fixed maximum length.
- **Icon:** `"icon"` gives an icon for the Launcher.
- **A program with many similar parts:** a Python script can write it. See `tools/mkbricks.py`, which writes `bricks.vbp` with 32 bricks.

## Examples

In `blocks/examples/`:

| File | Program |
|---|---|
| `pong.vbp` | PONG |

| | |
|---|---|
| `catch.vbp` | CATCH |
| `starcat.vbp` | STARCAT: costumes, a tune from a list |
| `counter.vbp` | COUNTER: a window and buttons |
| `bricks.vbp` | BRICKS (written by `tools/mkbricks.py`) |

The `*.vbt` files are Studio programs as text, for `tools/mkvbs.py`. See chapter 5.

## Studio or web editor?

| | **Studio** | **web editor** |
|---|---|---|
| Where | on the CPC | on the PC |
| File | `.VBS` | `.vbp` |
| Variables | 128, A–H15 | with names |
| Lists, noise, sprite size | no | yes |
| SAVE/LOAD VARS | yes | no |
| Text style, `.MUS` music | yes | yes |
| The program | needs all the memory | small, runs alongside others |

# 7. The V-OS SDK: programs in assembly

With this SDK you write your own programs in Z80 assembly. They run inside V-OS like DEMO or CLOCK: they appear in the Launcher, the Desktop and the Commander, and they have their own menu, buttons, dialogs, sound and files. They also run alongside other programs.

## Quick start

```
make new-app NAME=hello
```

This creates the folder `myapps/hello/` from `sdk/template/`. It has three files:

- `hello.asm`: the program. The arrows move a box, there is a BEEP button, a seconds counter, and its own menu (ABOUT, BACK) next to VIEW and HELP.
- `texts.txt`: its texts, in English and Greek.
- `icon.txt`: its icon, 16×16, for the Launcher.

```
make app NAME=hello
```

This builds `build/HELLO.BIN` and puts it on the programs' disk, `build/apps.dsk`. `make run` puts that in B:; in the Launcher, TAB shows B:.

```
make run
```

This opens the disk in Caprice32. HELLO is in the Launcher, on the last page.

- Plain `make` builds all the programs in `myapps/` and puts them on the disk, together with V-OS.
- The name has 1–8 lowercase letters or digits and starts with a letter. It cannot be the name of a V-OS program (e.g. demo).
- You need RASM, Python 3 and iDSK, as for all of V-OS.
- In Caprice32 you run the program yourself, from the Launcher.

A bigger program:

- Put more `.asm` files in its folder (e.g. `art.asm` with sprites) and bring them in with `include "art.asm"`. The build finds them, and rebuilds the program when they change.
- Whatever you put in the subfolder `disc/` (e.g. `disc/LEVEL1.DAT`) goes onto the disk as it is, for `VOS_LOAD`.
- With a subfolder `owndisc/`, the program also gets a disk of its own, `build/<name>.dsk`, with the `.BIN` and the files of `owndisc/`. It stays off `apps.dsk`. You put it in B: and run it from the Launcher (TAB, the other drive). Which drive it was started from, so that it loads its files from there, is in `VOS_SELF`: `db drive`, then its name (11 bytes).
- **A file to open (version 14):** when RETURN in the Desktop or the Commander opens a file in your program, `VOS_OPEN` (`#B000`) holds `db 1, drive, 11-byte name` (0: none). Read it in `init`, with A = 0 and also with A = 2 (the program was already running), then write 0 there. The types and their programs are in the kernel's `ft_open` table (`src/kernel/files_ext.asm`).
- A program with a relocation table (like those of the SDK) runs beside others when it is up to 12K; a bigger one (up to 16K) runs on its own without a memory expansion, like one without a table (MARKDOWN, READER). For the rest, load files into the data bank.

## How a program works

A program is a `.BIN`, built for `VOS_APP_ADDR` (`#4000`). It starts like this:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
        jp init                 ; +0
        jp event                ; +3  HL -> the event
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 the icon (optional)
```

- **`init`** sets mode, palette, menu and buttons, turns on `VE_FRAME` if needed, and draws everything.
  - ○ A = 0: a fresh start.
  - ○ A = 2: the program returns from a view or another program. Its state is as it left it; it sets the above again and draws.
- **`event`** is called for each event and returns immediately. A program never waits on its own (no wait loop). For motion or time, `VOS_TICKS` with A = 1, and then a `VE_FRAME` arrives 50 times a second.
- **`redraw`** redraws everything, e.g. when the language changes or the desk accessory closes.
- **Exit:** `VOS_EXIT`, which returns to the view that started it.

### The order inside `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; the frames
        push hl
        call VOS_UI_EVENT               ; menus, buttons, CONTROL+letter
        pop hl
        or a : jr nz,command            ; A = the command
        call VOS_MENU_OPEN : ret nz     ; menu open: the keys are its own
        ...                             ; your own keys and clicks
command: ...
        jp VOS_COMMAND                  ; whatever is not yours (VIEW, HELP)
```

**While a menu is open, do not draw.** Otherwise you will draw over it. On `VE_FRAME`, update the state, but draw only if `VOS_MENU_OPEN` gives Z. Dialogs (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) have their own loop, so no event arrives while they are open.

### The events

`HL` → 8 bytes: `db type, code, char, mods; dw x, y`.

| type | what | code / char / x, y |
|---|---|---|
| `VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2 | key | code = key number (`VK_*`, keyboard row·8 + bit), char = the character in the pack's language |
| `VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4 | pointer | x, y in pixels of the mode |
| `VE_FRAME` 8 | frame (with `VOS_TICKS`) | — |
| `VE_LANG` 7 | the keyboard language changed | — |

### Three kinds of program

- **With a relocation table (the SDK default):** `make app` builds it twice, at `#4000` and at `#4100`. `tools/mkreloc.py` adds the table from the difference.
  - It runs alongside other programs, in the 12K from `#5000`, and continues where it stopped (F6 shows the running programs).
  - Its data must be inside its own bytes (labels, `ds`), never at fixed addresses.
  - If `mkreloc.py` gives an error, an address is used somewhere in a way that does not survive relocation. E.g. `ld a,label & 255`, or only the low byte of an address.
- **Without a table:** it takes the whole bank (up to 16K, code and data), but runs only on its own, unless there is a memory expansion. This needs your own rule in the Makefile (see `STUDIO.BIN`).
- **Desk accessory:** up to 4K, with a table. It opens with F5 on top of anything, and `init` gets A = 1. See `apps/clock/clock.asm` and the description in `src/api/vos_api.inc`.

### Rules

- **Registers:** calls corrupt all registers except IX. Arguments go in A, BC, DE, HL (IX for `VOS_RECT` and `VOS_BLIT`). Results return in A or HL. On failure, carry is set and A = error number, which `VOS_ERROR` displays.
- **Memory:** do not page another bank in at `#4000`, because the program will vanish from under you. For 16K more there is the data bank (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). It is shared with views and other programs, and keeps its data only while the program is running. With a memory expansion (API 21) each program gets one of its own, which keeps what is in it until the program ends, whatever runs meanwhile; `VX_DATA` says which (A = 1: its own).
- **Coordinates:** x in pixels of the mode (mode 0: 0–191, mode 1: 0–383, mode 2: 0–767), y from 0 to 258. The menu bar takes y 0–11.
- **RASM:** labels are not case-sensitive (`BOX` and `box` are the same). Hex is written with `#` and not with `&`.

## The data formats

- **Menu bar** (`VOS_SET_BAR`): `db count; dw menu, ...`. The shared VIEW and HELP come from `VOS_MENU_VIEW` and `VOS_MENU_HELP`.
- **Menu:** `db title, count` and for each item `db text, command, letter, 0`.
  - text: `VOS_SEPARATOR` for a line
  - command: 1.., what `VOS_UI_EVENT` returns
  - letter: CONTROL+letter shortcut, or 0
- **Buttons** (`VOS_SET_WIDGETS`, then `VOS_DRAW_WIDGETS`): `db count`, and for each button `dw x, y; db width in characters, text, command, 0`.
- **Dialog** (`VOS_DIALOG` → A = button, 0 the first, ESC gives the last): `db title, text, number of buttons (1-3), button, ...`. For buttons, `VM_OK`, `VM_YES` and `VM_NO`.
- **Texts** (`VOS_TEXTS`): made by `tools/mktexts.py` from `texts.txt`, with lines `NAME | English | Greek`. Both columns are written in UTF-8 and become bytes of the V-OS character set (`tools/voscs.py`), which holds Greek and the Latin letters of French, Spanish, German and Italian.

    - It gives `T_NAME` (`VT_FIRST` = `#C0` and up) and the table `texts:`. Up to 64 texts.
    - A `|` with no spaces around it starts a new line in dialogs.
    - Texts go wherever a "text" is asked for (menus, buttons, dialogs). With `VOS_MSG` you get their address for `VOS_PRINT`.

- **Palette** (`VOS_PALETTE`): 17 firmware colors (0–26): 16 pens and the border. In mode 1, pens 0–3 count.
- **Rectangle** (`VOS_RECT`, IX →): `dw x, y, w, h; db pen`.
- **Icon:** 16 lines of 16 characters in `icon.txt`. Each character is a pen in hex (mode 0, Launcher palette) or `.` for transparent. The Launcher reads it from the disc: while the program runs, the 256 bytes after "IC" are free for buffers (PAINT keeps its rows there).
- **File names** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 bytes, `"NAME EXT"`. Drive 0 = A:, 1 = B:.

## The API (version 22)

| Call | Input → output | What it does |
|---|---|---|
| `VOS_EXIT` | — | end, back to the view that started it |
| `VOS_TICKS` | A = 1 / 0 (+ `VOS_PTR_KEYS`) | `VE_FRAME` 50 times a second, or none. With `VOS_PTR_KEYS` (2) the arrows and COPY also move the pointer and click |
| `VOS_FRAMES` | → HL (and A bits 16–23) | frames since startup |
| `VOS_RANDOM` | → A | 0–255 |
| `VOS_COMMAND` | A = command | the shared commands of VIEW and HELP (6–19). Number your own commands from `VC_APP` (32) up, or a VIEW or HELP item runs one of yours |
| `VOS_ERROR` | A = error | error dialog |
| `VOS_MODE` | A = 0 / 1 / 2 | mode 0 (192×259, 16 pens), 1 (384×259, 4 pens) or 2 (768×259, 2 pens, 96 text columns; version 25). In mode 2 menus and dialogs draw in pen 1 on pen 0: pen 0 white and pen 1 black give black text on white paper (NOTEPAD, MARKDOWN, READER, SHEET with ^M) |
| `VOS_PALETTE` | HL → 17 colors | |
| `VOS_CLS` | A = pen | clears the screen |
| `VOS_PLOT` | HL = x, DE = y, A = pen | one pixel |
| `VOS_RECT` | IX → rectangle | filled rectangle |
| `VOS_COLOURS` | A = text pen (+ `VOS_BOLD`, `VOS_ITALIC`, `VOS_UNDERLINE`), C = background | for `VOS_PRINT` and `VOS_CHAR`. The styles are made from the same font, for every language |
| `VOS_PRINT` | HL = x, DE = y, BC → text, 0 | |
| `VOS_CHAR` | HL = x, DE = y, A = character | |
| `VOS_MSG` | A = text → HL | the address of a text (`T_*` or a system one) |
| `VOS_SET_BAR`, `VOS_DRAW_BAR` | HL → menu bar | |

| | | |
|---|---|---|
| VOS_UI_EVENT | (the current event) → A | menu, button or shortcut command, 0 for none |
| VOS_DIALOG | HL → dialog → A | |
| VOS_INPUT | A = title, HL → text, C = maximum → carry if cancelled | text input (needs C + 2 bytes) |
| VOS_SET_WIDGETS, VOS_DRAW_WIDGETS | HL → buttons | |
| VOS_MENU_VIEW, VOS_MENU_HELP | → HL | the shared menus |
| VOS_CATALOG | A = drive → A = files, HL → list | 16 bytes each: 11 the name, at +12 the size in K, at +13 1 for a folder (drive 2). With A = drive + VOS_CAT_EXT and HL → "EXT", only the files with that extension |
| VOS_FILE_PICK | A = title → A = file, carry if cancelled | picker dialog, after VOS_CATALOG |
| VOS_LOAD | A = drive, HL → name, DE = where → BC = length | |
| VOS_SAVE | A = drive, HL → name, DE = from, BC = length | binary file, replacing the old one. With A + VOS_LOAD_IX (#40, version 20) its header says it loads at IX, not at DE (a .SCR: IX = #C000) |
| VOS_DELETE | A = drive, HL → name | |
| VOS_TEXTS | HL → text table | the program's texts |
| VOS_MENU_OPEN | → NZ if a menu is open | |
| VOS_KEY_HELD | A = key → NZ while pressed | for games |
| VOS_BLIT | IX → `dw x, y; db bytes, lines; dw source; db source stride` | image: pairs (mask, pixels) |
| VOS_SOUND | A = channel 0–2, HL = period (bit 15: noise), C = volume 0–15, B = frames | a note; 440 Hz is period 142 |
| VOS_SPLIT | HL → `dw y1, y2; db 8 colors` (0 = turns off) | mixed screen: mode 0 between y1 and y2 |
| VOS_RUN | A = drive, HL → name | runs another program, and then this one starts again |
| VOS_KEEP | → HL → 16 bytes | memory that stays between programs |
| VOS_BANK_GET, VOS_BANK_PUT | HL = from, DE = to, BC = bytes | copy from and to the data bank |
| VOS_BACKGROUND | A = 1 / 0 | keeps running or freezes when behind another |
| VOS_DRAG | HL = x, DE = y, BC = w, A = h → carry: new HL, DE | dragging a window by its title |
| VOS_FIT | same → HL, DE | brings a window inside the screen |
| VOS_EXT | A = a VX_* call, BC, DE, HL as it wants → A, HL, C, DE, carry | the calls after the table (version 15), below |

**Drives:** 0 = A:, 1 = B:, 2 = DISK, the SD card of an M4 board (version 28), in the folder the Desktop or the Commander last went into. File calls and VX_FREE work there as on a disc (8.3 names; without an M4 board: carry, A = 35). Databases are only on discs.

**The VOS_EXT calls:** VX_VERSION → A = API, HL = V-OS version (BCD). VX_FREE C = drive → A = free K.

Unknown: carry VX_CLIP_PUT HL = text, BC = bytes → the clipboard (up to 1K). VX_CLIP_GET DE = where, BC = room → HL = size (version 16). VX_M4 HL → a command for the M4 board (db size, dw command, data), DE → the response, BC = bytes; carry: no M4 board (version 17). VX_MEM → HL = the RAM in K, DE = the K free for programs (version 18). VX_BEHIND DE → db drive, 11-byte name of a file: its program (by type) starts behind this one, with init A = 3 and the file in VOS_OPEN (version 19; MUSIC knows it). VX_DATA → A = 1 if the data bank is the program's own (memory expansion), 0 if it is the shared bank 4 (version 21; NOTEPAD keeps text up to 16K this way). VX_DB C = a DB_* call, DE, HL → a database (version 22, below). VX_UTF8 HL → text, BC = bytes, E = 0 in the program's memory or VOS_DATA in its data bank: if it is UTF-8 (from a PC or the web), it becomes the V-OS character set there → BC = bytes now, A = 1; A = 0: it was not UTF-8 and stays as it was (version 23; NOTEPAD and READER do this). VX_CALC C = a CALC_* call, DE → a block, HL → text → formulas and numbers (version 24, below). VX_LANG → HL = the code of the language V-OS shows ("EN", "EL", "FR"...; L the first letter) (version 26; VBRUN shows the translations of Studio's programs this way). VX_IRQ HL = a routine of the program → the kernel calls it once a frame from the interrupt, in front or behind, with its bank there and interrupts off; 0 stops it, and so does the program's end; carry: not for an accessory or a program alone in bank 5 (version 27; MUSIC plays its songs this way, so they do not stop while another program loads or draws). It must be short, keep IY, EXX and AF', write only its own memory and the AY, and call nothing of V-OS.

## Databases (version 22)

VOS_EXT with A = VX_DB and C = a DB_* call works on a database: a file NAME.VDB of 1K pages, as big as the disc allows (up to 255K). It holds up to 8 tables; a table has up to 10 fields, text (up to 255 characters) or a number (2 bytes), and may have a key field (text up to 64 characters, or a number) with an index, so that FIND is fast. A record is its fields one after the other; its id (rid) is a 16-bit number. One database is open at a time, for the program that opened it. Each call writes its changes to the disc before it returns. The kernel loads the engine, DB.SYS, from drive A: the first time.

| Call | In → out |
|---|---|
| DB_CREATE, DB_OPEN | DE → db drive, 11-byte name ("STREETS VDB"); OPEN → A = 1: Latin-1 (of V-OS 0.15) |
| DB_CLOSE | |
| DB_TABLE | DE → name (8), fields, key field (0: none), then each field: name (8), type, length → A = table |
| DB_USE | E = the table the calls below use |
| DB_INFO | E = table, HL → 124 bytes: its descriptor → A = tables |
| DB_ADD | HL → record → DE = rid |
| DB_GET, DB_PUT | DE = rid, HL → the record |
| DB_DEL | DE = rid |
| DB_FIND | HL → key as in a record (room for a record), E = bytes that count (0: all) → DE = the first record with that key or after it |
| DB_FIRST, DB_LAST, DB_NEXT, DB_PREV | HL → room for the record, or 0 → DE = rid, in key order; carry at the end |
| DB_COUNT | → HL = records |

Errors: carry and A (DB_ERR_*, and the disc's errors), which VOS_ERROR shows. Text is kept as it is typed, in the V-OS character set, the same in every language: one database can hold Greek, French and German together. Only a database of V-OS 0.15 with Latin letters (Latin-1) is different: DB_OPEN then gives A = 1; show DB_ERR_ALPHABET with VOS_ERROR. Sorting and FIND compare bytes: in a DB_TEXT key capitals and small letters, and accented ones, are different. A DB_ALPHA field (type 2, from version 23) is text like DB_TEXT, but as a key it is sorted and found without case or accents:

ΕΛΕΝΗ, Ελένη and ελένη, or Élodie and ELODIE, are the same; the record keeps the text as it was typed. Examples are STREETS (`myapps/streets`), a street index, and DATABASE (`myapps/database`), a whole database program with tables, a list and a form; `tools/vdb.py` shows and checks a `.VDB` file on the PC.

### Formulas (version 24)

`VOS_EXT` with A = `VX_CALC` and C = a `CALC_*` call works out formulas, as a spreadsheet does. The kernel loads the engine, `CALC.SYS`, from drive A: the first time. A number is 32 bits, in hundredths (1234 is 12.34), from -21474836.46 to 21474836.46. A formula has `+ - * /`, brackets, numbers (`12`, `3.5`, `.25`; a third decimal rounds), cells (`A1` to `Z255`) and the functions SUM, AVG, MIN, MAX, COUNT (of values, cells and ranges such as `B2:D9`, between `,` or `;`), ABS(x), INT(x) (down to a whole number) and ROUND(x) or ROUND(x, 0–2), in capitals or not. Products and quotients are rounded to the hundredth.

DE → the program's block, `CB_SIZE` (10) bytes: `CB_RESULT` (a number, 4 bytes), `CB_COLS`, `CB_ROWS`, `CB_GRID` (dw), `CB_BANK`, `CB_AT`. The cells are a grid of the program's: `CB_COLS` × `CB_ROWS` numbers of 4 bytes, row after row, at `CB_GRID`, in its data bank when `CB_BANK` = `VOS_DATA`. A cell `CALC_NONE` has no number (empty, or text: 0 in `+ - * /`, left out by the functions); `CALC_BAD` has an error.

| Call | In → out |
|---|---|
| `CALC_EVAL` | HL → formula, 0 (= first or not) → `CB_RESULT` and DE:HL (DE high) |
| `CALC_NUMBER` | HL → text: only a number? → as `CALC_EVAL` |
| `CALC_TEXT` | `CB_RESULT` → text at HL (13 bytes of room: `-12.5`), A = its length |
| `CALC_CELL` | HL → text: a cell (`b12`) at its start? → D = column, E = row (from 0), A = its characters |

Errors: carry and A (`CALC_ERR_*`: 30 a wrong formula, 31 division by zero, 32 a number too big, 33 a cell outside the grid, 34 a cell it uses has an error, 29 `CALC.SYS` is not on A:), which `VOS_ERROR` shows; then `CB_RESULT` = `CALC_BAD` and `CB_AT` = where in the text. The example is SHEET (`myapps/sheet`), a spreadsheet that keeps its values in its data bank and works the formulas out again after each change.

Full details, e.g. on the background, the accessory and `VOS_SPLIT`, are in `src/api/vos_api.inc`. For examples, see `apps/demo/demo.asm` (mode 0, motion), `apps/clock/clock.asm` (accessory) and `apps/mixed/mixed.asm` (mixed screen). A whole game with sprites, a stage from a file and sound is `myapps/vrouves/`.

An AI assistant writing a program should read `AI.md`: the same rules, shorter, with the traps of RASM and of the data bank.

## Testing

`make test-sdk` builds the template as TEMPLATE.BIN and opens it from the Launcher in Caprice32, with key presses defined by `src/kernel/test_script.asm` (script 15). It takes screenshots in `build/shots`: its screen, the box after it moves, its menu, the ABOUT dialog, and the return to the Launcher.

For your own automated test, copy script 15 and the `test-sdk` target of the Makefile.

# 8. Developing V-OS

For those who work on V-OS itself. Programs *for* V-OS are described in the SDK.

## What you need

- RASM, the assembler
- iDSK, for the disk images
- Python 3, for the tools. `make docs` also needs `markdown` and `pymupdf`.
- GNU make 4.3 or newer
- for testing, Caprice32 (snap `caprice32`)

## Build

| Command | What it does |
|---|---|
| `make` | `build/vos.dsk` (A:): the kernel, the views, the languages, DEMO, MIXED, STUDIO, CLOCK and the `.VBS` sources; `build/apps.dsk` (B:): the other programs and `myapps/*`; and the disc of each program with `owndisc/`. If a file does not fit on a real disc (180K), it stops with an error |
| `make run` | opens the disk in Caprice32 (`RUN"VOS"`) |
| `make shot` | starts without a window and takes a screenshot |
| `make docs` | the manual as PDFs: `docs/V-OS.pdf` (Greek) and `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it` |
| `make windows` | `build/V-OS-windows.zip`: `V-OS.exe` for Windows, with CPCEMU 3.0.2 (CPC 6128, 576K, AMX mouse, M4 board), `vos.dsk` in A:, `apps.dsk` in B: and the programs' own discs beside them. V-OS starts by itself. Needs WSL and the portable CPCEMU zip (`CPCEMU_ZIP=...`) |
| `make linux` | `build/V-OS-linux.tar.gz`: the `V-OS` script for Linux (x86_64), with CPCEMU 2.5 (3.0.2 is not out for Linux) and the same settings and discs. Needs SDL2 on the system and CPCEMU's Linux tar.gz (`CPCEMU_TGZ=...`) |
| `make songs` | the songs of Arkos Tracker 3 as `.MUS` files in `myapps/music/owndisc` (`tools/mkmusic.py`; needs Arkos Tracker 3, `ARKOS=...`) |
| `make new-app NAME=x`, `make app NAME=x` | your own program (SDK) |
| `make clean` | deletes `build/` |

## Repo layout

| Folder | Contents |
|---|---|
| `src/vos.asm` | the root of the build: loader and kernel, writing to the disk |
| `src/boot/` | the loader that AMSDOS runs at `#4000`, RAM expansion detection |
| `src/kernel/` | vectors, interrupts, file system, views, API (`api.asm`), settings, kernel extension (`ext.asm`), test scripts (`test_script.asm`) |
| `src/drivers/` | Gate Array, CRTC, keyboard, AMX mouse, FDC, AY |
| `src/gfx/` | overscan graphics, text, mixed screen |
| `src/ui/` | menus, buttons, dialogs, text input, file picker |

| | |
|---|---|
| src/shell/ | the views as overlays: Desktop, Launcher, Commander, Setup |
| src/api/ vos_api.inc | the API for programs |
| lang/ | the languages (texts, keyboards, fonts) |
| apps/ | the programs in assembly: DEMO, MIXED, STUDIO, CLOCK, and their icons |
| blocks/ | the web editor, vbc.py, runtime.asm, the Studio player (player.asm), examples |
| sdk/ | the SDK: guide and program template |
| tools/ | tools: fonts, languages, texts, icons, relocation, .VBS from text, screenshots, PDF |
| docs/ | this manual in Greek (manual/) and in English, French, Spanish, German and Italian (manual/en/, fr/, es/, de/, it/), the PDFs, the requirements (REQUIREMENTS.md) |

## Memory, in brief

| Area | What it holds |
|---|---|
| #0000–#3DFF | the kernel. In page zero, after the restart vectors, the API table at #0108 |
| up to #4000 | the kernel stack |
| #4000–#7FFF | the bank "window" |
| #8000–#FFFF | the 32K overscan screen. In its gaps, kernel buffers |

The banks that go into the window:

| Bank | What it holds |
|---|---|
| 4 | file and load buffer, and the data bank of the running program |
| 5 | the view at #4000–#4FFF, and from #5000 the programs with a relocation table (or one program without a table in the whole bank) |
| 6 | whatever the menus and dialogs hide, and from #7000 the desk accessory |
| 7 | the language pack, the Commander lists, the English texts, and from #7000 the kernel extension |

With a memory expansion, its banks take programs. The details, address by address, are in docs/REQUIREMENTS.md (§4.2 and in each "Υλοποίηση M..." (implementation) section).

## Conventions

- **RASM:** hex is written #C000 and not &C000, because RASM reads & as AND. Labels are not case-sensitive.
- **Comments:** in English, short, as in the existing code. Each file starts with a block that says what it does.
- **Documentation:** in Greek. MILESTONES.md (English), MILESTONES.el.md (Greek) and docs/REQUIREMENTS.md have a section for each milestone. The manual is written in Greek and translated into the other five languages of V-OS.
- **Texts:** the system texts are in lang/*/messages.txt, and the programs' texts in texts.txt (for tools/mktexts.py).

## Testing

The tests run V-OS in Caprice32 without a window and take screenshots in build/shots/<delay>/.

**How they work:**

- Caprice32 without a window cannot press arrows, COPY or f keys. That is why the key presses are made by a special build: a -DTEST_SCRIPT=n puts script n into src/kernel/

`test_script.asm`, a table of keys per frame.

- Each `make test-…` target builds a disk with its script and takes a screenshot at each delay in `…_SHOTS`.
- The delay is measured in "Z" presses: about 2.5–3.5 frames each. The mapping is not fixed, so you catch a moment with two or three delays around it.
- Once V-OS writes to the disk, Caprice32 does not close by itself, and the test ends with a timeout. This is expected in test-vars, test-costume and others.
- You look at the screenshots yourself, e.g. combined into one image with `ffmpeg … hstack/vstack`.

| Target | What it tests |
|---|---|
| `test-input` | keys, Studio: blocks, files, running |
| `test-calc` | CALC with clicks, then `CALC.VBS` in Studio and its title in English and French (^T) |
| `test-costume` | costume in Studio, running, reopening |
| `test-acc` | the clock as a desk accessory |
| `test-multi`, `test-switch`, `test-expand`, `test-behind` | several programs, F6, memory expansion, background |
| `test-drag` | dragging windows |
| `test-langs`, `test-lang` | the languages, saving the language |
| `test-mines`, `test-bricks` | the games |
| `test-vars` | the 128 variables, SAVE/LOAD VARS |
| `test-sdk` | the SDK template |
| `test-apps` | the two discs as given out, `vos.dsk` (with the test keys) in A: and `apps.dsk` in B:: the Launcher shows B: and PONG runs from there |
| `test-vrouves` | VROUVES from its disc in B:, through the Launcher (TAB): building, wave 1, the funds, HIGH SCORES |
| `test-notepad` | NOTEPAD: typing, DEL, the save dialog, OPEN with only `.TXT` files; then ^M: the page (mode 2) |
| `test-notepad-big` | NOTEPAD's text window in a Z80 emulator (`tools/notepad_check.py`): a 12K file with its own data bank, moving, typing and SAVE, text up to 16K, and 4K on a 6128 |
| `test-data` | (576K) NOTEPAD with a 15K text down to line 33, MARKDOWN meanwhile, then NOTEPAD back up to the start: the lines from its own data bank |
| `test-paint` | PAINT: line, box, fill, the menus, then the layers (hiding one), OTHER DRIVE and EXPORT .SCR to B: ("SAVED AS .SCR") |
| `test-paint-big` | PAINT on the whole screen (576K): lines, the window moved by its bars (a click and a drag), its place, saved as `B:NONAME.SCR` and opened again |
| `test-scr` | PAINT's EXPORT .SCR (its code in a Z80 emulator, `tools/scr_check.py`) loaded in BASIC from a disc, with the palette; every pixel of the screen checked |
| `test-about` | HELP > ABOUT V-OS |
| `test-music` | MUSIC from `music.dsk` in B:: a song plays, and goes on behind the Launcher |
| `test-markdown` | MARKDOWN from `markdown.dsk` in B:: `README.MD` opened, the page with ^P, a screen down, back to the text; then ^M: the page (mode 2) |
| `test-open` | opening by type in the Commander: `README.MD` in MARKDOWN from B:, a `.TXT` in NOTEPAD from A: |

| | |
|---|---|
| `test-sysinfo` | SYSINFO from the Commander: version, API, memory, free K on A: and B:, ^R, ^Q. With `SYSINFO_RAM=576`, with a 512K expansion |
| `test-clip` | the clipboard: selecting with SHIFT, ^C and ^V in NOTEPAD, ^A, ^X, ^V, then ^V in MARKDOWN |
| `test-webget` | by hand, in the CPCEMU of `make linux` (M4 board; needs a screen and the internet): WEBGET downloads `example.com/index.html`. Close CPCEMU with F12 when SAVED shows, and `INDEX.HTM` is looked for on the disc |
| `test-reader` | READER from `reader.dsk` in B: opens `INDEX.MD`, TAB selects a link, RETURN opens `ABOUT.MD`, DEL goes back, and an `https://` link gives a message; then ^M: the page (mode 2) |
| `test-db` | the database engine in a Z80 emulator (`tools/db_check.py`): thousands of random calls on four tables, checked against a model, the file page by page (`tools/vdb.py`), until the disc is full |
| `test-db-disc` | `DB.SYS` with the kernel's file system on a DSK image (`tools/db_disc_check.py`) |
| `test-streets` | STREETS from `streets.dsk` in B:: a new `STREETS.VDB` with 24 streets, finding, a new street |
| `test-database-z80` | DATABASE in a Z80 emulator, with the database in Python and the screen as text: two tables, 47 records from the form, the order without case or accents, finding, a change, a deletion, scrolling |
| `test-database` | DATABASE in Caprice32: a new database on B:, a table, three records from the form, the list, finding, a change |
| `test-calc-sys` | the formula engine `CALC.SYS` in a Z80 emulator (`tools/calc_check.py`): fixed formulas and 40,000 random ones, numbers and texts, checked against a model in Python |
| `test-sheet-z80` | SHEET in a Z80 emulator with `CALC.SYS` itself and the screen as text: values, `#ERR`, cells that use each other, scrolling, SAVE and OPEN, a UTF-8 file, `BUDGET.VSH`, a full sheet; then ^M: the page (mode 2) |
| `test-sheet` | SHEET in Caprice32: numbers and a formula typed, `CALC.SYS` from A:, the sheet saved on B:; then ^M: the page (mode 2) |
| `test-kbds` | KEYBOARDS: ALL SIX in Setup, then in NOTEPAD the `:` and Z keys with each keyboard: `:z`, `:ζ`, `mw`, `ñz`, `öy`, `òz` |
| `test-charset`, `test-reader-utf8` | UTF-8 files in Greek, French, Spanish, German and Italian in NOTEPAD and in READER |
| `test-utf8` | the code of `VX_UTF8` in a Z80 emulator, against `tools/voscs.py` |
| `test-styles` | STYLES.VBN (`blocks/examples/styles.vbt`) from the Commander: plain, bold, italic, underlined and all three, with the STYLE block |
| `test-music-block` | with 576K: TUNE.VBN (`blocks/examples/tune.vbt`) starts MUSIC (from B:) behind it with AGANAMEM.MUS; F6 shows "> TUNE" and MUSIC, and RETURN brings MUSIC to the front: A:AGANAMEM, PLAYING. With `MBLOCK_RAM=128`: "CLOSE ANOTHER PROGRAM FIRST" |
| `test-amsdos`, `test-reboot`, `test-system` | V-OS files in BASIC, reboot, SYSTEM disk |

**New script:**

1. Add an `IF TEST_SCRIPT == n` at the start of the chain in `test_script.asm`, with `ELSE` before the next one and one more `ENDIF` at the end.
2. Add a target to the Makefile, like `test-sdk`.

Each key is written as `dw from, to : db row, bit` of the keyboard. The key number is row·8 + bit, e.g. RETURN = 18 = row 2, bit 2.

**Without an emulator:** a program's code also runs on a Z80 inside Python (the `z80` package). You load the `.BIN` at `#4000` and make the API calls return immediately (or do whatever is needed, e.g. files in a dictionary). Then you call init and event and read the memory. It is much faster than screenshots for errors in numbers, files and jumps. The label addresses come from `rasm … -s -sl` (a `.sym` file).

## Every milestone

1. Code, and a test in Caprice32: new or existing, with screenshots.
2. The tests the change touches.
3. A section in `MILESTONES.md` and `MILESTONES.el.md`, and a "Υλοποίηση Μ…" (implementation) section in `docs/REQUIREMENTS.md`.
4. **Update the manual in all six languages** (`docs/manual/` in Greek, `docs/manual/<code>/` for en, fr, es, de, it, and `sdk/README.md`, `sdk/README.<code>.md` if the API changes), then `make docs` for all the PDFs.
5. Commit and push.